

CSCI 455 Robotics Studio

Note: All active robots have a new cardboard box with a lidar sitting on the robot. The standoffs to mount the lidar will be here this week, so in the meantime just set the lidar on a flat surface of the robot for developing and testing. **BE VERY CAREFUL** when wiring the LIDAR the wires from the USB board to the lidar are very flimsy and the wires will pull out of the clip harnesses if you are not careful. I had to wiggle my connections to get them to make the proper connection. When the lidar is powered it should automatically start spinning, don't block the spinning. The lidar does not come with the USB cable so I will distribute those also.

Project 3: Lidar Safety Stop with Flask Remote Control

Assigned: March 23rd

Demo Days: Monday March 30th and Wednesday April 1 (Friday is holiday)

Overview

In this project, you will extend your existing Flask-based remote control system by integrating your Lidar sensor into the driving logic.

Your robot should still accept joystick commands from the Flask web interface, but driving must now be **safety-aware**:

- If the user commands the robot to move **forward**, the robot must ignore that command when an obstacle is too close in front.
- If the user commands the robot to move **backward**, the robot must ignore that command when an obstacle is too close behind.
- Turning in place or steering motions that do not drive into the obstacle may still be allowed, depending on your implementation.

The goal is to add a simple real-time safety layer between joystick input and wheel motion.

Learning Objectives

By completing this project, you will:

- integrate sensor input into an existing control system

- use Lidar data to detect obstacles in specific directions and ignore the parts of the robot that will be blocking the lidar.
- combine Flask, joystick input, and robot motion in a safe way
- design clean software structure between web control, robot logic, and sensor logic
- test and debug a real-time robotics safety feature

Project Requirements

You must use your **existing Flask remote control server** from the first project.

Your robot must:

1. **Accept joystick input from the Flask interface**
2. **Read Lidar data continuously while the program runs**
3. **Determine whether an obstacle is blocking travel**
4. **Ignore unsafe joystick commands**
5. **Allow safe joystick commands to continue working normally**
 1. If the forward progress is blocked by an obstacle the robot should not move, but when the obstacles is moved or leaves the robot should continue in the proper direction.

Required Behavior

1. Forward Safety Stop

When the joystick requests forward movement, the robot must check whether an obstacle is within a dangerous distance in front of the robot.

- If the front is clear, the robot may move forward.
- If the front is blocked, the robot must not move forward.
- The includes dynamic obstacles that move into the robot's view as it is moving.

2. Backward Safety Stop

When the joystick requests backward movement, the robot must check whether an obstacle is within a dangerous distance behind the robot.

- If the rear is clear, the robot may move backward.

- If the rear is blocked, the robot must not move backward.
- Dynamic obstacles as in the forward direction movement.
-

Lidar Zones (the lidar orientation should have 359, 0, 1 directly in front of the robot)

You must define at least two detection zones:

- **Front danger zone**
- **Rear danger zone**

For example, you might define:

- Front zone = angles near 0 degrees
- Rear zone = angles near 180 degrees

You may choose the exact angular ranges and stopping distances, but they must be reasonable and clearly documented. And safe for the robot.

Example:

- Front blocked if any reading from 330-360 degrees or 0-30 degrees is under 800 mm
- Rear blocked if any reading from 150-210 degrees is under 800 mm

You may tune these values for your robot.

Program Structure

Your code should be organized so that:

- the **Flask server** receives joystick commands
- the **Lidar system** updates current obstacle status
- the **robot driving code** makes the final decision about whether motion is allowed

Minimum Technical Expectations

Your project must include:

- continuous Lidar scanning while Flask is running
- some shared state or flags indicating front/rear blocked status

- logic that prevents unsafe motion commands from reaching the motors
- printed debug output showing what the robot thinks is blocked
- a working live demo on the real robot

Deliverables

- Your work journal will be turned in like the first two projects.

Demo Requirements

During your demo, you must show all of the following:

1. **Normal driving when no obstacle is present**
2. **Forward blocked by an object**
 1. This should include an object moving in range stopping the robot
 2. Object walks out of blocking range and the robot continues (forward and backwards) as long as the joystick is still engaged.
3. **Backward blocked by an object**
4. **Safe motion still works in an unblocked direction**
5. **Terminal output showing the safety logic responding**

Grading (100 points + 25points journal)

Functionality - 60 points

- 20pts: Flask joystick control still works
- 20 pts: Forward motion is blocked correctly
- 20 pts: Backward motion is blocked correctly

Lidar Integration - 10 points

- 5 pts: Lidar runs continuously while the system is active
- 5 pts: Front/rear obstacle detection is correctly implemented

Demo / Documentation - 30 points

- 20 pts: Successful live demo

Total: 100 points

Hints

- Start by getting your Lidar code running by itself first.
- Print the scan angles and distances so you know which direction is front and which is rear on your robot.
- Keep your first version simple.
- It is often easiest to let the Lidar thread update boolean flags like:
 - front_blocked
 - rear_blocked
- Then check those flags before sending wheel commands.
- Be careful about thread safety and shared variables.
- Test with slow robot motion first.

This project has a quick due date, only one week before demos are Monday and Wednesday of next week.